
Introduction to Database
CS350

Project

Deadline: Monday 09/08/2026 @ 23:59

[Total Mark is 20]

Student Details:

CRN:50241

Name: Nawaf Kamal Allahyani

ID: 240006019

Name: Abdullellah Khalid Alali

ID: 240059980

Name: MOHAMMED ALSHAMMASI

ID: 240050332

Name: Abdulwahid Alqatari

ID: 240045553

Instructions:

- You must submit two separate copies (**one Word file and one PDF file**) using the Assignment Template on Blackboard via the allocated folder. These files **must not be in compressed format**.
- It is your responsibility to check and make sure that you have uploaded both the correct files.
- Zero mark will be given if you try to bypass the SafeAssign (e.g. misspell words, remove spaces between words, hide characters, use different character sets, **convert text into image** or languages other than English or any kind of manipulation).
- Email submission will not be accepted.
- You are advised to make your work clear and well-presented. This includes filling your information on the cover page.
- You must use this template, failing which will result in zero mark.
- You **MUST** show all your work, and text must not be converted into an image, unless specified otherwise by the question.
- Late submission will result in ZERO mark.
- The work should be your own, copying from students or other resources will result in ZERO mark.
- Use **Times New Roman** font for all your answers.

Project Instructions

- You can work on this project as a group (minimum 3 and maximum 4 students). Each group member must submit the project individually with all group member names mentioned on the cover page.
- This project **worth 20 marks** and will be distributed as in the following:
 - Database Design. (6 marks)
 - Database Implementation. (6 marks)
 - SQL Operations. (6 marks)
 - Project Reflection. (2 marks)
- Each leader in the group must submit one report about their chosen Project via the Blackboard (**Email submission will not be accepted and will be awarded ZERO marks**) containing the following:
 - A. Task 1 : database design
 - B. Task 2: database implementation
 - C. All requested queries/results
 - D. Screenshots from MySQL (or any other software you use) of **all the tables** after population and **query results**.
 - E. Reflection (300–500 words) describing the team's experience, challenges, design decisions, and AI usage (if applicable).
- Submit one **SQL file** containing the complete database implementation, including database creation, tables, data insertion, required queries, VIEW, and TRIGGER.
- Submit a **GitHub** (or GitLab) repository containing the complete project with regular commits and meaningful commit messages.
- Submit a **progress report** describing the completed work, key decisions, challenges, future plan, and contribution of each group member.
- Include **links** to the live report document, GitHub/GitLab repository, and Google Colab notebook (if applicable) in the **Project Artifacts** section of the final report.
- You are advised to make your work clear and well presented; marks may be reduced for poor presentation. This includes filling in your information on the cover page.
- You **MUST** show all your work, and text **must not** be converted into an image unless specified otherwise by the question.
- **Late submission will result in ZERO marks being awarded.**
- The work should be your own. Copying **from students or other resources, including AI software, will result in ZERO marks.**

*Learning**Outcome(s):**CLO1, CLO2, CLO3,**CLO4, CLO5*

Project Description

Design and Implementation of a Smart Clinic Database System

A private clinic currently manages its patient information manually, making it difficult to organize appointments, treatments, and payments. The clinic has decided to develop a database system to improve data organization and retrieval.

Your team is required to design, implement, and test a complete relational database using the concepts learned throughout this course. The project should demonstrate your understanding of database design, ER/EER modeling, relational mapping, SQL implementation, and advanced SQL queries. The project must be completed using MySQL.

Task 1 – Database Design (6 Marks)

- Design a complete database for the clinic that includes at least six entities such as Patients, Doctors, Appointments, Treatments, Medicines, Payments, or any other appropriate entities.
- A complete ER/EER diagram.
- Primary and foreign keys.
- Appropriate relationships with cardinality.
- At least one specialization/generalization (EER feature).
- Any assumptions made during the design process.

Entities are:

1. Patient.
2. Department.
3. Employee (Supertype).
4. Doctor (Subtype).
5. Nurse (Subtype).
6. Pharmacist (Subtype).
7. Staff (Subtype).
8. Room.
9. Appointment.
10. Treatment.
11. Prescription.
12. Medicine.
13. Payment.

Entities and their attributes:

Entity	Attributes	Primary Key (PK)	Foreign Key (FK)
Patient	PatientID, FirstName, LastName, DateOfBirth, Gender, Phone, Email, Address, BloodType, EmergencyContact	PatientID	
Department	DepartmentID, DepartmentName, Location, PhoneExtension	DepartmentID	
Employee (Supertype)	EmployeeID, DepartmentID, FirstName, LastName, Gender, DateOfBirth, Phone, Email, HireDate, Salary	EmployeeID	DepartmentID (DepartmentID)
Doctor (Subtype)	EmployeeID, RoomID, LicenseNumber, Qualification, Specialty, ConsultationFee	Doctor.EmployeeID	EmployeeID (Employee) RoomID (Room)
Nurse (Subtype)	EmployeeID, RoomID, Qualification, Shift,	Nurse.EmployeeID	EmployeeID (Employee) RoomID (Room)
Pharmacist (Subtype)	EmployeeID, RoomID, LicenseNumber, Qualification,	Pharmacist.EmployeeID	EmployeeID (Employee) RoomID (Room)
Staff (Subtype)	EmployeeID, RoomID, JobTitle.	Staff.EmployeeID	EmployeeID (Employee) RoomID (Room)
Room	RoomID, DepartmentID, RoomNumber, RoomType, Floor	RoomID	DepartmentID (Department)

Appointment	AppointmentID, PatientID, DoctorID, AppointmentDate, AppointmentTime, Status, Reason	AppointmentID	PatientID (PatientID). DoctorID (Doctor.EmployeeID).
Treatment	TreatmentID, AppointmentID, TreatmentName, Diagnosis, Notes	TreatmentID	AppointmentID (Appointment).
Prescription	PrescriptionID, TreatmentID, MedicineID, Dosage, Duration	PrescriptionID	TreatmentID (Treatment). MedicineID (Medicine).
Medicine	MedicineID, MedicineName, Category, UnitPrice, StockQuantity, ExpiryDate	MedicineID	
Payment	PaymentID, AppointmentID, Amount, PaymentDate, PaymentMethod, PaymentStatus	PaymentID	AppointmentID (Appointment).

Relationships and Cardinality:

The first entity	Relationship	The second entity	Cardinality
Department	Employs	Employee	1 : M
Department	Contain	Room	1 : M
Employee	ISA	Doctor (Specialization)	1 : 1
Employee	ISA	Nurse (Specialization)	1 : 1
Employee	ISA	Pharmacist (Specialization)	1 : 1
Employee	ISA	Staff (Specialization)	1 : 1
Room	HAS	Doctor	1 : 1
Room	HAS	Nurse	1 : 1
Room	HAS	Pharmacist	1 : 1
Room	HAS	Staff	1 : 1
Patient	Books	Appointment	1 : M
Doctor	Responsible for	Appointment	1 : M
Appointment	Results In	Treatment	1 : M
Treatment	Includes	Prescription	1 : M
Medicine	References	Prescription	1 : M
Appointment	Generates	Payment	1 : 1

Specialization:

The database we designed use Specialization (ISA) to model different employee roles. It is Total and Disjoint specialization.

Supertype: Employee.

The Employee supertype stores the common information shared by all clinic employees, which are:

EmployeeID, DepartmentID, FirstName, LastName, Gender, DateOfBirth, Phone, Email, HireDate, Salary.

Subtypes: Doctor, Nurse, Pharmacist and staff.

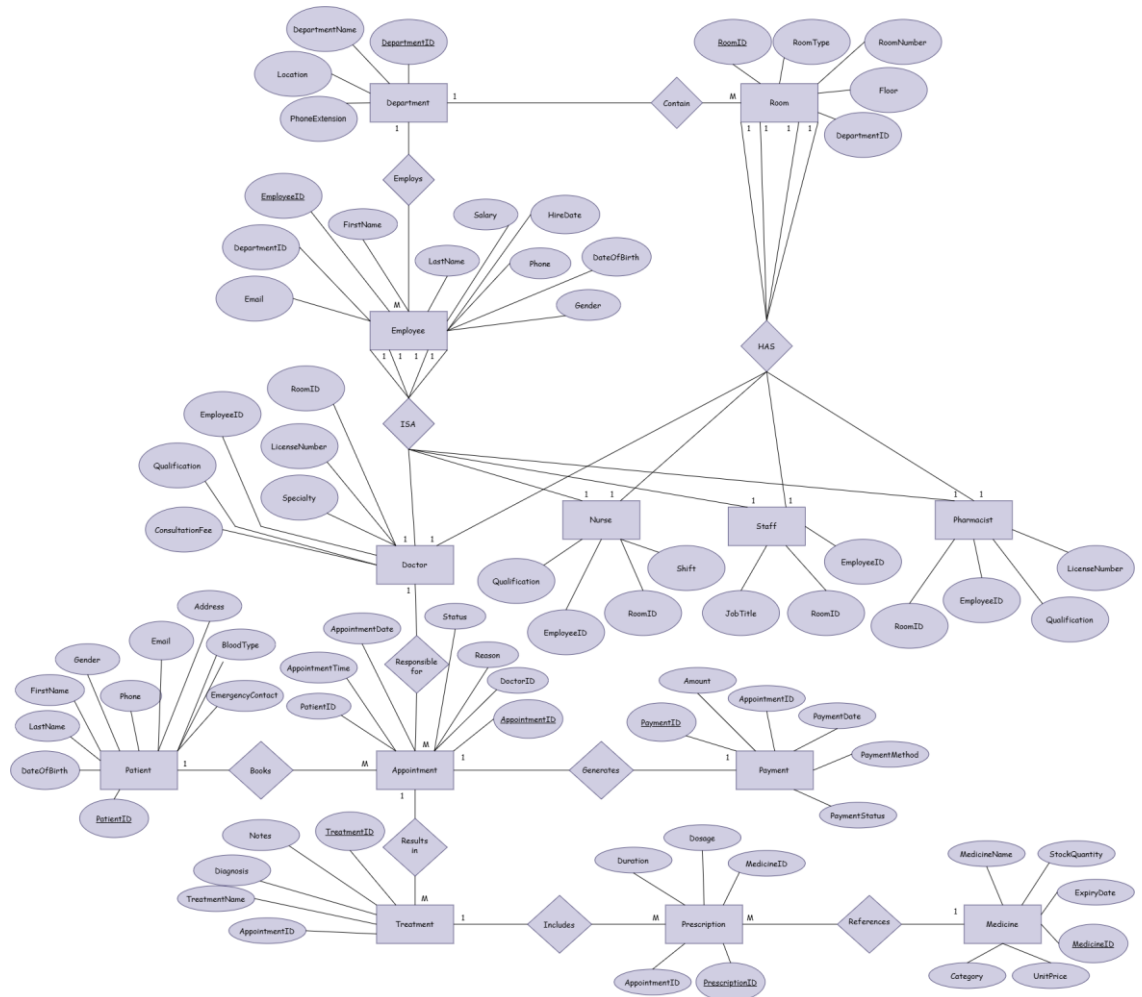
- Doctor has these additional attributes:
RoomID, LicenseNumber, Qualification, Specialty, ConsultationFee.
- Nurse has these additional attributes:
RoomID, Shift, Qualification.
- Pharmacist has these additional attributes:
RoomID, LicenseNumber, Qualification.
- Staff has this additional attribute:
RoomID, JobTitle.

This specialization removes duplicate employee data while permitting each staff position to preserve its unique characteristics.

Assumptions:

- Each patient is uniquely identified by a PatientID.
- Each employee belongs to exactly one specific department.
- Employees mapped under the Doctor subtype can be assigned to conduct patient appointments.
- A patient can book and have multiple appointments over time.
- A doctor can be responsible for and conduct multiple appointments.
- One single appointment can result in or include multiple individual treatments.:
One medical treatment sequence can generate multiple separate prescriptions.
- Each prescription record references exactly one single medicine code.
- One specific medicine from the inventory can appear across many different patient prescriptions.
- Every individual appointment generates exactly one corresponding payment record.
- Medicines are strictly tracked in a centralized pharmacy inventory table before being mapped to active patient prescriptions.
- Each doctor, nurse, pharmacist, or staff member must be linked to exactly one record in the Employee supertype table, and no single employee can hold more than one specific role simultaneously (Total and Disjoint).

ER diagram:



Task 2 – Database Implementation (6 Marks)

Complete the following tasks using MySQL. Include screenshots of both your SQL code and the corresponding output (results) for each task:

- Create the database and all required tables.
- Use appropriate data types and constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, etc.).
- Insert at least 5 records into each main table.

These screenshots show the process of creating our Database with the tables (include the code and the output):

```

1  -- =====
2  -- Smart Clinic Database System
3  -- =====
4  |
5  CREATE DATABASE SmartClinicDB;
6  USE SmartClinicDB;

```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0012 seconds.)

```
-- ===== -- Smart Clinic Database System -- ===== CREATE DATABASE
SmartClinicDB;
```

[Edit inline] [Edit] [Create PHP code]

⚠ Error: #1046 No database selected

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0002 seconds.)

```
USE SmartClinicDB;
```

[Edit inline] [Edit] [Create PHP code]

⚠ Error: #1046 No database selected

```

1  -- Create table Department
2
3  CREATE TABLE Department (
4      DepartmentID INT PRIMARY KEY AUTO_INCREMENT,
5      DepartmentName VARCHAR(50) NOT NULL UNIQUE,
6      Location VARCHAR(100) NOT NULL,
7      PhoneExtension VARCHAR(10) NOT NULL
8  );
9
10
11 -- Create table Patient
12
13 CREATE TABLE Patient (
14     PatientID INT PRIMARY KEY AUTO_INCREMENT,
15     FirstName VARCHAR(30) NOT NULL,
16     LastName VARCHAR(30) NOT NULL,
17     Gender ENUM('Male','Female') NOT NULL,
18     DateOfBirth DATE NOT NULL,
19     Phone VARCHAR(10) NOT NULL UNIQUE,
20     Email VARCHAR(50) NOT NULL UNIQUE,
21     Address VARCHAR(100),
22     BloodType VARCHAR(5),
23     EmergencyContact VARCHAR(50)
24 );
25

```

Task 2 – Database Implementation (6 Marks)

```

26 -- Create table Employee
27
28 CREATE TABLE Employee (
29     EmployeeID INT PRIMARY KEY AUTO_INCREMENT,
30     DepartmentID INT NOT NULL,
31     FirstName VARCHAR(30) NOT NULL,
32     LastName VARCHAR(30) NOT NULL,
33     Gender ENUM('Male','Female') NOT NULL,
34     DateOfBirth DATE NOT NULL,
35     Phone VARCHAR(10) NOT NULL UNIQUE,
36     Email VARCHAR(50) NOT NULL UNIQUE,
37     HireDate DATE NOT NULL,
38     Salary DECIMAL(10,2) NOT NULL CHECK (Salary > 0),
39     FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
40         ON UPDATE CASCADE ON DELETE RESTRICT
41 );
42
43 -- Create table Room
44
45 CREATE TABLE Room (
46     RoomID INT PRIMARY KEY AUTO_INCREMENT,
47     DepartmentID INT NOT NULL,
48     RoomNumber VARCHAR(10) NOT NULL UNIQUE,
49     RoomType VARCHAR(30) NOT NULL,
50     Floor INT NOT NULL,
51     FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
52         ON UPDATE CASCADE ON DELETE RESTRICT
53 );
54
55 -- Create table Doctor
56
57 CREATE TABLE Doctor (
58     EmployeeID INT PRIMARY KEY,
59     RoomID INT NOT NULL UNIQUE,
60     LicenseNumber VARCHAR(30) NOT NULL UNIQUE,
61     Qualification VARCHAR(50) NOT NULL,
62     Specialty VARCHAR(50) NOT NULL,
63     ConsultationFee DECIMAL(10,2) NOT NULL,
64     FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID)
65         ON UPDATE CASCADE ON DELETE CASCADE,
66     FOREIGN KEY (RoomID) REFERENCES Room(RoomID)
67         ON UPDATE CASCADE ON DELETE RESTRICT
68 );
69
70 -- Create table Nurse
71
72 CREATE TABLE Nurse (
73     EmployeeID INT PRIMARY KEY,
74     RoomID INT NOT NULL UNIQUE,
75     Shift ENUM('Morning','Evening','Night') NOT NULL,
76     Qualification VARCHAR(50) NOT NULL,
77     FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID)
78         ON UPDATE CASCADE ON DELETE CASCADE,
79     FOREIGN KEY (RoomID) REFERENCES Room(RoomID)
80         ON UPDATE CASCADE ON DELETE RESTRICT
81 );
82
83 -- Create table Pharmacist
84
85 CREATE TABLE Pharmacist (
86     EmployeeID INT PRIMARY KEY,
87     RoomID INT NOT NULL UNIQUE,
88     LicenseNumber VARCHAR(30) NOT NULL UNIQUE,
89     Qualification VARCHAR(50) NOT NULL,
90     FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID)
91         ON UPDATE CASCADE ON DELETE CASCADE,
92     FOREIGN KEY (RoomID) REFERENCES Room(RoomID)
93         ON UPDATE CASCADE ON DELETE RESTRICT
94 );
95
96 -- Create table Staff
97
98 CREATE TABLE Staff (
99     EmployeeID INT PRIMARY KEY,
100     RoomID INT NOT NULL UNIQUE,
101     JobTitle VARCHAR(50) NOT NULL,
102     FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID)
103         ON UPDATE CASCADE ON DELETE CASCADE,
104     FOREIGN KEY (RoomID) REFERENCES Room(RoomID)
105         ON UPDATE CASCADE ON DELETE RESTRICT
106 );
107

```

```

108 -- Create table Appointment
109
110 CREATE TABLE Appointment (
111     AppointmentID INT PRIMARY KEY AUTO_INCREMENT,
112     PatientID INT NOT NULL,
113     DoctorID INT NOT NULL, -- references Doctor(EmployeeID)
114     AppointmentDate DATE NOT NULL,
115     AppointmentTime TIME NOT NULL,
116     Status ENUM('Scheduled','Completed','Cancelled') NOT NULL DEFAULT 'Scheduled',
117     Reason VARCHAR(150),
118     FOREIGN KEY (PatientID) REFERENCES Patient(PatientID)
119     ON UPDATE CASCADE ON DELETE CASCADE,
120     FOREIGN KEY (DoctorID) REFERENCES Doctor(EmployeeID)
121     ON UPDATE CASCADE ON DELETE RESTRICT
122 );
123
124 -- Create table Treatment
125
126 CREATE TABLE Treatment (
127     TreatmentID INT PRIMARY KEY AUTO_INCREMENT,
128     AppointmentID INT NOT NULL,
129     TreatmentName VARCHAR(100) NOT NULL,
130     Diagnosis VARCHAR(100) NOT NULL,
131     Notes TEXT,
132     FOREIGN KEY (AppointmentID) REFERENCES Appointment(AppointmentID)
133     ON UPDATE CASCADE ON DELETE CASCADE
134 );

```

```

136 -- Create table Medicine
137
138 CREATE TABLE Medicine (
139     MedicineID INT PRIMARY KEY AUTO_INCREMENT,
140     MedicineName VARCHAR(50) NOT NULL UNIQUE,
141     Category VARCHAR(50) NOT NULL,
142     UnitPrice DECIMAL(8,2) NOT NULL CHECK (UnitPrice >= 0),
143     StockQuantity INT NOT NULL DEFAULT 0 CHECK (StockQuantity >= 0),
144     ExpiryDate DATE NOT NULL
145 );
146
147
148 -- Create table Prescription
149
150 CREATE TABLE Prescription (
151     PrescriptionID INT PRIMARY KEY AUTO_INCREMENT,
152     TreatmentID INT NOT NULL,
153     MedicineID INT NOT NULL,
154     Dosage VARCHAR(30) NOT NULL,
155     Duration VARCHAR(30) NOT NULL,
156     FOREIGN KEY (TreatmentID) REFERENCES Treatment(TreatmentID)
157     ON UPDATE CASCADE ON DELETE CASCADE,
158     FOREIGN KEY (MedicineID) REFERENCES Medicine(MedicineID)
159     ON UPDATE CASCADE ON DELETE RESTRICT
160 );
161
162 -- Create table Payment
163
164 CREATE TABLE Payment (
165     PaymentID INT PRIMARY KEY AUTO_INCREMENT,
166     AppointmentID INT NOT NULL UNIQUE,
167     Amount DECIMAL(10,2) NOT NULL CHECK (Amount >= 0),
168     PaymentDate DATE NOT NULL,
169     PaymentMethod ENUM('Cash','Card','Insurance') NOT NULL,
170     PaymentStatus ENUM('Paid','Pending','Refunded') NOT NULL DEFAULT 'Pending',
171     FOREIGN KEY (AppointmentID) REFERENCES Appointment(AppointmentID)
172     ON UPDATE CASCADE ON DELETE CASCADE
173 );

```

Task 2 – Database Implementation (6 Marks)

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0024 seconds.)

```
-- Create table Department CREATE TABLE Department ( DepartmentID INT PRIMARY KEY AUTO_INCREMENT, DepartmentName VARCHAR(50) NOT NULL UNIQUE, Location VARCHAR(100) NOT NULL, PhoneExtension VARCHAR(10) NOT NULL );
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
-- Create table Patient CREATE TABLE Patient ( PatientID INT PRIMARY KEY AUTO_INCREMENT, FirstName VARCHAR(30) NOT NULL, LastName VARCHAR(30) NOT NULL, Gender ENUM('Male','Female') NOT NULL, DateOfBirth DATE NOT NULL, Phone VARCHAR(10) NOT NULL UNIQUE, Email VARCHAR(50) NOT NULL UNIQUE, Address VARCHAR(100), BloodType VARCHAR(5), EmergencyContact VARCHAR(50) );
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0003 seconds.)

```
-- Create table Employee CREATE TABLE Employee ( EmployeeID INT PRIMARY KEY AUTO_INCREMENT, DepartmentID INT NOT NULL, FirstName VARCHAR(30) NOT NULL, LastName VARCHAR(30) NOT NULL, Gender ENUM('Male','Female') NOT NULL, DateOfBirth DATE NOT NULL, Phone VARCHAR(10) NOT NULL UNIQUE, Email VARCHAR(50) NOT NULL UNIQUE, HireDate DATE NOT NULL, Salary DECIMAL(10,2) NOT NULL CHECK (Salary > 0), FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID) ON UPDATE CASCADE ON DELETE RESTRICT );
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0003 seconds.)

```
-- Create table Room CREATE TABLE Room ( RoomID INT PRIMARY KEY AUTO_INCREMENT, DepartmentID INT NOT NULL, RoomNumber VARCHAR(10) NOT NULL UNIQUE, RoomType VARCHAR(30) NOT NULL, Floor INT NOT NULL, FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID) ON UPDATE CASCADE ON DELETE RESTRICT );
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
-- Create table Doctor CREATE TABLE Doctor ( EmployeeID INT PRIMARY KEY, RoomID INT NOT NULL UNIQUE, LicenseNumber VARCHAR(30) NOT NULL UNIQUE, Qualification VARCHAR(50) NOT NULL, Specialty VARCHAR(50) NOT NULL, ConsultationFee DECIMAL(10,2) NOT NULL, FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID) ON UPDATE CASCADE ON DELETE CASCADE, FOREIGN KEY (RoomID) REFERENCES Room(RoomID) ON UPDATE CASCADE ON DELETE RESTRICT );
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
-- Create table Nurse CREATE TABLE Nurse ( EmployeeID INT PRIMARY KEY, RoomID INT NOT NULL UNIQUE, Shift ENUM('Morning','Evening','Night') NOT NULL, Qualification VARCHAR(50) NOT NULL, FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID) ON UPDATE CASCADE ON DELETE CASCADE, FOREIGN KEY (RoomID) REFERENCES Room(RoomID) ON UPDATE CASCADE ON DELETE RESTRICT );
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
-- Create table Pharmacist CREATE TABLE Pharmacist ( EmployeeID INT PRIMARY KEY, RoomID INT NOT NULL UNIQUE, LicenseNumber VARCHAR(30) NOT NULL UNIQUE, Qualification VARCHAR(50) NOT NULL, FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID) ON UPDATE CASCADE ON DELETE CASCADE, FOREIGN KEY (RoomID) REFERENCES Room(RoomID) ON UPDATE CASCADE ON DELETE RESTRICT );
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0005 seconds.)

```
-- Create table Staff CREATE TABLE Staff ( EmployeeID INT PRIMARY KEY, RoomID INT NOT NULL UNIQUE, JobTitle VARCHAR(50) NOT NULL, FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID) ON UPDATE CASCADE ON DELETE CASCADE, FOREIGN KEY (RoomID) REFERENCES Room(RoomID) ON UPDATE CASCADE ON DELETE RESTRICT );
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
-- Create table Appointment CREATE TABLE Appointment ( AppointmentID INT PRIMARY KEY AUTO_INCREMENT, PatientID INT NOT NULL, DoctorID INT NOT NULL, -- references Doctor(EmployeeID) AppointmentDate DATE NOT NULL, AppointmentTime TIME NOT NULL, Status ENUM('Scheduled','Completed','Cancelled') NOT NULL DEFAULT 'Scheduled', Reason VARCHAR(150), FOREIGN KEY (PatientID) REFERENCES Patient(PatientID) ON UPDATE CASCADE ON DELETE CASCADE, FOREIGN KEY (DoctorID) REFERENCES Doctor(EmployeeID) ON UPDATE CASCADE ON DELETE RESTRICT );
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
-- Create table Treatment CREATE TABLE Treatment ( TreatmentID INT PRIMARY KEY AUTO_INCREMENT, AppointmentID INT NOT NULL, TreatmentName VARCHAR(100) NOT NULL, Diagnosis VARCHAR(100) NOT NULL, Notes TEXT, FOREIGN KEY (AppointmentID) REFERENCES Appointment(AppointmentID) ON UPDATE CASCADE ON DELETE CASCADE );
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
-- Create table Medicine CREATE TABLE Medicine ( MedicineID INT PRIMARY KEY AUTO_INCREMENT, MedicineName VARCHAR(50) NOT NULL UNIQUE, Category VARCHAR(50) NOT NULL, UnitPrice DECIMAL(8,2) NOT NULL CHECK (UnitPrice >= 0), StockQuantity INT NOT NULL DEFAULT 0 CHECK (StockQuantity >= 0), ExpiryDate DATE NOT NULL );
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
-- Create table Prescription CREATE TABLE Prescription ( PrescriptionID INT PRIMARY KEY AUTO_INCREMENT, TreatmentID INT NOT NULL, MedicineID INT NOT NULL, Dosage VARCHAR(30) NOT NULL, Duration VARCHAR(30) NOT NULL, FOREIGN KEY (TreatmentID) REFERENCES Treatment(TreatmentID) ON UPDATE CASCADE ON DELETE CASCADE, FOREIGN KEY (MedicineID) REFERENCES Medicine(MedicineID) ON UPDATE CASCADE ON DELETE RESTRICT );
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
-- Create table Payment CREATE TABLE Payment ( PaymentID INT PRIMARY KEY AUTO_INCREMENT, AppointmentID INT NOT NULL UNIQUE, Amount DECIMAL(10,2) NOT NULL CHECK (Amount >= 0), PaymentDate DATE NOT NULL, PaymentMethod ENUM('Cash','Card','Insurance') NOT NULL, PaymentStatus ENUM('Paid','Pending','Refunded') NOT NULL DEFAULT 'Pending', FOREIGN KEY (AppointmentID) REFERENCES Appointment(AppointmentID) ON UPDATE CASCADE ON DELETE CASCADE );
```

[Edit inline] [Edit] [Create PHP code]

INSERT sample data into tables:

```
1  -- INSERT sample data
2  |
3  INSERT INTO Department (DepartmentName, Location, PhoneExtension) VALUES
4  ('Cardiology', 'Building A, 2nd Floor', '101'),
5  ('Pediatrics', 'Building A, 1st Floor', '102'),
6  ('Orthopedics', 'Building B, 3rd Floor', '103'),
7  ('Dermatology', 'Building B, 2nd Floor', '104'),
8  ('General Medicine', 'Building A, 1st Floor', '105');
9
10
11 INSERT INTO Patient
12 (FirstName, LastName, Gender, DateOfBirth, Phone, Email, Address, BloodType, EmergencyContact)
13 VALUES
14 ('Ahmed', 'alfahid', 'Male', '1990-05-15', '0501110001', 'ahmed.alfahid@gmail.com', 'King Salman, Riyadh', 'O+', '0502220001'),
15 ('Fatima', 'alharbi', 'Female', '1985-08-22', '0501110002', 'fatima.alharbi@gmail.com', 'Almalga, Riyadh', 'A+', '0502220002'),
16 ('Mohammed', 'alotaibi', 'Male', '1978-12-03', '0501110003', 'mohammed.alotaibi@gmail.com', 'Alnaseem, Riyadh', 'B+', '0502220003'),
17 ('Sara', 'alghamdi', 'Female', '1995-07-19', '0501110004', 'sara.alghamdi@gmail.com', 'Alsafart, Riyadh', 'AB+', '0502220004'),
18 ('Noura', 'alsaud', 'Female', '2000-01-30', '0501110005', 'noura.alsaud@gmail.com', 'Alsafart, Riyadh', 'O-', '0502220005');
19
20
21 INSERT INTO Employee
22 (FirstName, LastName, Gender, DateOfBirth, Phone, Email, HireDate, Salary, DepartmentID)
23 VALUES
24
25 ('Saud', 'alghamdi', 'Male', '1980-03-10', '0551000001', 'saud.alghamdi@Nawafclinic.com', '2015-06-01', 22000.00, 1),
26 ('Maha', 'alqahtani', 'Female', '1982-07-25', '0551003002', 'maha.alqahtani@Nawafclinic.com', '2016-09-15', 21000.00, 2),
27 ('Abdullah', 'alharbi', 'Male', '1979-11-02', '0551000003', 'abdullah.alharbi@Nawafclinic.com', '2014-04-10', 23000.00, 3),
28 ('Sara', 'almutairi', 'Female', '1985-02-14', '0551001104', 'sara.almutairi@Nawafclinic.com', '2017-12-01', 20500.00, 4),
29 ('Faisal', 'alzahrani', 'Male', '1983-09-09', '0551000005', 'faisal.alzahrani@Nawafclinic.com', '2018-01-20', 24000.00, 5),
30
31 ('Laila', 'alshammari', 'Female', '1990-04-18', '0551004506', 'laila.alshammari@Nawafclinic.com', '2020-03-01', 12000.00, 1),
32 ('Omar', 'alotaibi', 'Male', '1988-06-30', '0551000332', 'omar.alotaibi@Nawafclinic.com', '2019-07-15', 11500.00, 2),
33 ('Reem', 'alharbi', 'Female', '1992-10-12', '0551000008', 'reem.alharbi@Nawafclinic.com', '2021-02-20', 11800.00, 3),
34 ('Khalid', 'alsubaie', 'Male', '1986-12-05', '0551000009', 'khalid.alsubaie@Nawafclinic.com', '2018-08-01', 12200.00, 4),
35 ('Hanan', 'alenezi', 'Female', '1991-05-22', '0551000010', 'hanan.alenezi@Nawafclinic.com', '2020-11-10', 12000.00, 5),
36
37 ('Naser', 'aldosari', 'Male', '1984-08-19', '0551000011', 'naser.aldosari@Nawafclinic.com', '2016-05-05', 14000.00, 1),
38 ('Mona', 'alrasheed', 'Female', '1987-03-27', '0551000012', 'mona.alrasheed@Nawafclinic.com', '2017-09-12', 13800.00, 2),
39 ('Turki', 'alshammari', 'Male', '1982-07-07', '0551000013', 'turki.alshammari@Nawafclinic.com', '2015-11-01', 14500.00, 3),
40 ('Huda', 'aljuhani', 'Female', '1993-01-11', '0551000014', 'hoda.aljuhani@Nawafclinic.com', '2019-04-18', 13500.00, 4),
41 ('Salman', 'alkhalidi', 'Male', '1989-09-15', '0551000015', 'salman.alkhalidi@Nawafclinic.com', '2020-02-28', 14200.00, 5),
42
43 ('Amal', 'albalawi', 'Female', '1992-06-21', '0551067016', 'amal.albalawi@Nawafclinic.com', '2021-01-10', 9000.00, 1),
44 ('Fahad', 'alldhib', 'Male', '1988-10-02', '0551080047', 'fahad.alldhib@Nawafclinic.com', '2019-08-14', 9500.00, 2),
45 ('Nawal', 'Alsubaie', 'Female', '1990-04-09', '0551202018', 'nawal.alsubaie@Nawafclinic.com', '2020-05-20', 8800.00, 3),
46 ('Majed', 'alhumaidi', 'Male', '1985-12-25', '0541000019', 'majed.alhumaidi@Nawafclinic.com', '2017-07-01', 9700.00, 4),
47 ('Jawhar', 'Alenezi', 'Female', '1994-02-28', '0557089020', 'jawhar.alenezi@Nawafclinic.com', '2022-03-15', 9200.00, 5);
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74 INSERT INTO Pharmacist (EmployeeID, RoomID, LicenseNumber, Qualification) VALUES
75 (11, 1, 'LIC-P001', 'BPharm, Licensed Pharmacist'),
76 (12, 2, 'LIC-P002', 'BPharm, Licensed Pharmacist'),
77 (13, 3, 'LIC-P003', 'PharmD, Clinical Pharmacy'),
78 (14, 4, 'LIC-P004', 'BPharm, Licensed Pharmacist'),
79 (15, 5, 'LIC-P005', 'PharmD, Clinical Pharmacy');
80
81
82 INSERT INTO Staff (EmployeeID, RoomID, JobTitle) VALUES
83 (16, 1, 'Receptionist'),
84 (17, 2, 'Administrative Assistant'),
85 (18, 3, 'Medical Records Clerk'),
86 (19, 4, 'Billing Specialist'),
87 (20, 5, 'Patient Coordinator');
88
89
90 INSERT INTO Appointment
91 (PatientID, DoctorID, AppointmentDate, AppointmentTime, Status, Reason)
92 VALUES
93 (1, 1, '2026-05-01', '09:00:00', 'Completed', 'Chest pain follow-up'),
94 (2, 2, '2026-06-07', '10:30:00', 'Completed', 'Routine child vaccination'),
95 (3, 3, '2026-07-08', '11:15:00', 'Completed', 'Knee pain assessment'),
96 (4, 4, '2026-08-04', '13:00:00', 'Scheduled', 'Skin rash examination'),
97 (5, 5, '2026-09-01', '14:45:00', 'Scheduled', 'General health checkup');
98
99
```

```
100 INSERT INTO Treatment
101 (AppointmentID, TreatmentName, Diagnosis, Notes)
102 VALUES
103 (1, 'Medication Therapy', 'Stable Angina', 'Prescribed cholesterol-lowering medication and lifestyle modifications. Schedule follow-up after two weeks.'),
104 (2, 'Immunization', 'Routine Immunization', 'MMR vaccine administered successfully. Patient observed for 15 minutes with no adverse reactions.'),
105 (3, 'Pain Management', 'Knee Osteoarthritis', 'Prescribed pain relief medication and physiotherapy sessions. Daily stretching exercises recommended.'),
106 (4, 'Topical Treatment', 'Eczema', 'Topical corticosteroid cream prescribed. Avoid skin irritants and moisturize regularly.'),
107 (5, 'General Checkup', 'General Health Assessment', 'Routine physical examination completed. No abnormal findings.');
```

```
108
109
110 INSERT INTO Medicine
111 (MedicineName, Category, UnitPrice, StockQuantity, ExpiryDate)
112 VALUES
113 ('Atorvastatin 20mg', 'Cardiovascular', 25.50, 200, '2027-12-31'),
114 ('MMR Vaccine', 'Immunization', 95.00, 80, '2026-11-30'),
115 ('Ibuprofen 400mg', 'Analgesic', 8.75, 500, '2028-06-15'),
116 ('Hydrocortisone Cream', 'Dermatology', 18.00, 150, '2027-08-20'),
117 ('Multivitamin Tablets', 'Nutritional', 15.00, 250, '2028-01-10');
```

```
118
119
120 INSERT INTO Prescription
121 (TreatmentID, MedicineID, Dosage, Duration)
122 VALUES
123 (1, 1, '1 tablet', '30 days'),
124 (2, 2, '0.5 mL injection', 'Single dose'),
125 (3, 3, '1 tablet', '14 days'),
126 (4, 4, 'Apply a thin layer', '14 days'),
127 (5, 5, '1 tablet', '30 days');
```

```
128
129
130 INSERT INTO Medicine
131 (MedicineName, Category, UnitPrice, StockQuantity, ExpiryDate)
132 VALUES
133 ('Atorvastatin 20mg', 'Cardiovascular', 25.50, 200, '2027-12-31'),
134 ('MMR Vaccine', 'Immunization', 95.00, 80, '2026-11-30'),
135 ('Ibuprofen 400mg', 'Analgesic', 8.75, 500, '2028-06-15'),
136 ('Hydrocortisone Cream', 'Dermatology', 18.00, 150, '2027-08-20'),
137 ('Multivitamin Tablets', 'Nutritional', 15.00, 250, '2028-01-10');
```

```
138
139
140 INSERT INTO Prescription
141 (TreatmentID, MedicineID, Dosage, Duration)
142 VALUES
143 (1, 1, '1 tablet', '30 days'),
144 (2, 2, '0.5 mL injection', 'Single dose'),
145 (3, 3, '1 tablet', '14 days'),
146 (4, 4, 'Apply a thin layer', '14 days'),
147 (5, 5, '1 tablet', '30 days');
```

```
148
149
150 INSERT INTO Payment
151 (AppointmentID, Amount, PaymentDate, PaymentMethod, PaymentStatus)
152 VALUES
153 (1, 250.00, '2026-05-02', 'Insurance', 'Paid'),
154 (2, 120.00, '2026-06-07', 'Cash', 'Paid'),
155 (3, 300.00, '2026-07-08', 'Card', 'Pending'),
156 (4, 180.00, '2026-08-04', 'Insurance', 'Paid'),
157 (5, 100.00, '2026-09-01', 'Card', 'Paid');
```

Note: all the code provided with this file.

Task 3 – SQL Operations (6 Marks)

Complete the following tasks using MySQL. For each task, include screenshots of both your SQL code and the corresponding output (results), along with a brief description (one to two sentences) explaining the purpose of the query and the information it is intended to retrieve.

- Write SELECT statements.
- Write JOIN queries.
- Write nested queries.
- Use aggregate functions with GROUP BY.
- Write UPDATE and DELETE statements.
- Create one VIEW.
- Create one TRIGGER.

SELECT statements:

Showing rows 0 - 4 (5 total, Query took 0.0006 seconds.)

SELECT * FROM patient;

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	PatientID	FirstName	LastName	Gender	DateOfBirth	Phone	Email	Address	BloodType	EmergencyContact
☐ Edit Copy Delete	1	Ahmed	alfahid	Male	1990-05-15	0501110001	ahmed.alfahid@gmail.com	King Salman, Riyadh	O+	0502220001
☐ Edit Copy Delete	2	Fatima	alharbi	Female	1985-08-22	0501110002	fatima.alharbi@gmail.com	Almarga, Riyadh	A+	0502220002
☐ Edit Copy Delete	3	Mohammed	alotaibi	Male	1978-12-03	0501110003	mohammed.alotaibi@gmail.com	Alnaseem, Riyadh	B+	0502220003
☐ Edit Copy Delete	4	Sara	alghamdi	Female	1995-07-19	0501110004	sara.alghamdi@gmail.com	Alsafat, Riyadh	AB+	0502220004
☐ Edit Copy Delete	5	Noura	alsaud	Female	2000-01-30	0501110005	noura.alsaud@gmail.com	Alsafat, Riyadh	O-	0502220005

This query retrieves all records from the Patient table. It is used to display complete patient information stored in the database.

```
SELECT EmployeeID, DepartmentID, Phone FROM Employee;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	EmployeeID	DepartmentID	Phone
☐ Edit Copy Delete	1	1	0551000001
☐ Edit Copy Delete	2	2	0551003002
☐ Edit Copy Delete	3	3	0551000003
☐ Edit Copy Delete	4	4	0551001104
☐ Edit Copy Delete	5	5	0551000005
☐ Edit Copy Delete	6	1	0551004506
☐ Edit Copy Delete	7	2	0551000332
☐ Edit Copy Delete	8	3	0551000008
☐ Edit Copy Delete	9	4	0551000009
☐ Edit Copy Delete	10	5	0551000010
☐ Edit Copy Delete	11	1	0551000011
☐ Edit Copy Delete	12	2	0551000012
☐ Edit Copy Delete	13	3	0551000013
☐ Edit Copy Delete	14	4	0551000014
☐ Edit Copy Delete	15	5	0551000015
☐ Edit Copy Delete	16	1	0551067016
☐ Edit Copy Delete	17	2	0551080047

This query retrieves (EmployeeID, DepartmentID, Phone) records from the employee table. It is used to display some employee information stored in the database.

```
SELECT appointmentID, appointmentTime, appointmentDate, STATUS FROM appointment WHERE STATUS = "Scheduled";
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	appointmentID	appointmentTime	appointmentDate	STATUS
☐ Edit Copy Delete	4	13:00:00	2026-08-04	Scheduled
☐ Edit Copy Delete	5	14:45:00	2026-09-01	Scheduled

This query retrieves appointments that are currently scheduled. It helps identify upcoming patient appointments.

JOIN queries:

```
SELECT Patient.FirstName, patient.LastName, appointment.AppointmentDate, appointment.AppointmentTime, appointment.Status FROM patient JOIN appointment ON patient.PatientID = appointment.PatientID;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

FirstName	LastName	AppointmentDate	AppointmentTime	Status
Ahmed	alfahid	2026-05-01	09:00:00	Completed
Fatima	alharbi	2026-06-07	10:30:00	Completed
Mohammed	alotaibi	2026-07-08	11:15:00	Completed
Sara	alghamdi	2026-08-04	13:00:00	Scheduled
Noura	alsaud	2026-09-01	14:45:00	Scheduled

This query joins the Patient and Appointment tables to display each patient's appointment date and appointment status.

Showing rows 0 - 4 (5 total, Query took 0.0010 seconds.)

```
SELECT employee.FirstName, employee.LastName, staff.JobTitle FROM employee JOIN staff ON employee.EmployeeID = staff.EmployeeID;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

FirstName	LastName	JobTitle
Amal	albalawi	Receptionist
Fahad	alduh	Administrative Assistant
Nawal	Alsubaie	Medical Records Clerk
Majed	alhumaidi	Billing Specialist
Jawhar	Alenezi	Patient Coordinator

This query joins the Employee and Staff tables to display staff names along with their JobTitle.

Nested queries:

```
SELECT MedicineName, UnitPrice FROM medicine WHERE UnitPrice = ( SELECT MAX(UnitPrice) FROM medicine );
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

MedicineName	UnitPrice
MMR Vaccine	95.00

This nested query retrieves the medicine with the highest unit price.

```
SELECT FirstName, LastName FROM employee WHERE Salary= ( SELECT MAX(Salary) FROM employee );
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	FirstName	LastName
☐ Edit <input type="image"/> Copy <input type="image"/> Delete	Faisal	alzahrani

This nested query retrieves the employee with the highest salary.

Aggregate functions with GROUP BY:

Showing rows 0 - 4 (5 total, Query took 0.0023 seconds.)

```
SELECT d.DepartmentName, AVG(e.salary) AS TheAverageSalary FROM department d JOIN employee e ON d.DepartmentID = e.DepartmentID GROUP BY d.DepartmentID, d.DepartmentName;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

DepartmentName	TheAverageSalary
Cardiology	14250.000000
Pediatrics	13950.000000
Orthopedics	14525.000000
Dermatology	13975.000000
General Medicine	14850.000000

This query groups employees by department and calculates the average salary in each department. It helps management analyze staffing levels and salary distribution across departments.

UPDATE statement:

`SELECT * FROM appointment;`

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	AppointmentID	PatientID	DoctorID	AppointmentDate	AppointmentTime	Status	Reason
☐ Edit Copy Delete	1	1	1	2026-05-01	09:00:00	Completed	Chest pain follow-up
☐ Edit Copy Delete	2	2	2	2026-06-07	10:30:00	Completed	Routine child vaccination
☐ Edit Copy Delete	3	3	3	2026-07-08	11:15:00	Completed	Knee pain assessment
☐ Edit Copy Delete	4	4	4	2026-08-04	13:00:00	Scheduled	Skin rash examination
☐ Edit Copy Delete	5	5	5	2026-09-01	14:45:00	Scheduled	General health checkup

✓ 1 row affected. (Query took 0.0003 seconds.)

`Update appointment SET status = 'Cancelled' where appointmentID = 4;`

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

✓ Showing rows 0 - 4 (5 total, Query took 0.0003 seconds.)

`SELECT * FROM appointment;`

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	AppointmentID	PatientID	DoctorID	AppointmentDate	AppointmentTime	Status	Reason
☐ Edit Copy Delete	1	1	1	2026-05-01	09:00:00	Completed	Chest pain follow-up
☐ Edit Copy Delete	2	2	2	2026-06-07	10:30:00	Completed	Routine child vaccination
☐ Edit Copy Delete	3	3	3	2026-07-08	11:15:00	Completed	Knee pain assessment
☐ Edit Copy Delete	4	4	4	2026-08-04	13:00:00	Cancelled	Skin rash examination
☐ Edit Copy Delete	5	5	5	2026-09-01	14:45:00	Scheduled	General health checkup

This query updates the status of Appointment 4 from Scheduled to Cancelled after the patient cancel. The SELECT statement verifies that the update was applied successfully.

DELETE statement:

✓ Showing rows 0 - 4 (5 total, Query took 0.0003 seconds.)

```
SELECT * FROM payment;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

				PaymentID	AppointmentID	Amount	PaymentDate	PaymentMethod	PaymentStatus
☐	Edit	Copy	Delete	1	1	250.00	2026-05-02	Insurance	Paid
☐	Edit	Copy	Delete	2	2	120.00	2026-06-07	Cash	Paid
☐	Edit	Copy	Delete	3	3	300.00	2026-07-08	Card	Pending
☐	Edit	Copy	Delete	4	4	180.00	2026-08-04	Insurance	Paid
☐	Edit	Copy	Delete	5	5	100.00	2026-09-01	Card	Paid

☐ Check all | With selected: Edit Copy Delete Export

✓ 1 row deleted. (Query took 0.0003 seconds.)

```
DELETE FROM payment WHERE PaymentID = 5;
```

[Edit inline] [Edit] [Create PHP code]

✓ Showing rows 0 - 3 (4 total, Query took 0.0003 seconds.)

```
SELECT * FROM payment;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

				PaymentID	AppointmentID	Amount	PaymentDate	PaymentMethod	PaymentStatus
☐	Edit	Copy	Delete	1	1	250.00	2026-05-02	Insurance	Paid
☐	Edit	Copy	Delete	2	2	120.00	2026-06-07	Cash	Paid
☐	Edit	Copy	Delete	3	3	300.00	2026-07-08	Card	Pending
☐	Edit	Copy	Delete	4	4	180.00	2026-08-04	Insurance	Paid

This query removes an incorrect payment record from the Payment table. The SELECT statement confirms that the record has been deleted.

VIEW:

```
1 CREATE VIEW patientappointments AS
2 SELECT
3     p.patientID,
4     CONCAT(p.FirstName, ' ', p.LastName) AS patientFullName,
5     a.AppointmentDate,
6     a.AppointmentTime,
7     a.STATUS
8 FROM patient p
9 JOIN appointment a ON p.PatientID = a.PatientID;
```

Table	Action	Rows	Type	Collation	Size	Overhead
☐ patientappointments	Browse Structure Search Insert Edit Drop	~0	View	---	-	-
1 table	Sum	~0	InnoDB	utf8mb4_general_ci	0 B	0 B
	☐ Check all	With selected:				

This view combines patient information with appointment details, making it faster to retrieve appointment schedules.

TRIGGER:

```
1 DELIMITER $$
2
3 CREATE TRIGGER before_appointment_insert
4 BEFORE INSERT ON appointment
5 FOR EACH ROW
6 BEGIN
7     IF NEW.AppointmentDate < CURDATE() THEN
8         SIGNAL SQLSTATE '45000'
9         SET MESSAGE_TEXT = 'The date could not be in the past!';
10    END IF;
11 END $$
12
13 DELIMITER ;
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0123 seconds)

```
CREATE TRIGGER before_appointment_insert BEFORE INSERT ON appointment FOR EACH ROW BEGIN IF NEW.AppointmentDate < CURDATE() THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'The date could not be in the past!'; END IF; END;
```

[\[Edit inline\]](#) [\[Edit\]](#) [\[Create PHP code\]](#)

```
1 INSERT INTO appointment (PatientID, DoctorID, AppointmentDate, AppointmentTime, STATUS)
2 VALUES (1, 1, '2024-02-09', '13:00:00', 'Pending'); -- test the TRIGGER |
```

Error

SQL query: [Copy](#)

```
INSERT INTO appointment (PatientID, DoctorID, AppointmentDate, AppointmentTime, STATUS)
VALUES (1, 1, '2024-02-09', '13:00:00', 'Pending');
```

MySQL said:

#1644 - The date could not be in the past!

This trigger automatically prevents inserting a new appointment if the appointment date is set in the past.

Task 4 – Reflection (2 Marks)

Write a **300–500-word reflection** on your experience while completing this project by answering the following questions. Your responses should be clear, concise, and supported with specific examples from your own work.

1. Describe one specific challenge or obstacle you encountered during this project and explain in detail how you identified and resolved it.
2. Explain why you chose your specific approach/method/model over at least one alternative you considered, and what tradeoffs informed that decision.
3. If you used any AI tools during this project (e.g., ChatGPT, Claude, Copilot), disclose which tool(s), for what specific purpose (e.g., debugging, grammar checking, brainstorming), and how you verified or modified the output.
4. What would you do differently if you were to redo this project with more time or resources?

The hardest issue regarding the current project we had to deal with is connected with the creation of database schema before writing SQL queries. In the beginning, it was difficult for us to understand how these entities are related and what to do with the supertype Employee and its subtypes. We carefully analyzed project requirements and some different models of the database and created ER diagram accordingly.

Then, we created tables using MySQL and checked them several times. Whenever there were any issues, for example, incorrect references between tables or wrong data samples, we fixed them and ran the script once again until everything worked.

The structure of the database is connected with our choice to use Enhanced EntityRelationship (EER) model with specialization (ISA) instead of having four separates and unrelated tables.

This design was considered due to its convenience regarding the number of relations that it contains, as well as the duplication of common properties of employees like name, phone number, email, salary, and date of hiring.

Supertypes and subtypes helped to make the database more normalized and easier to maintain. The only disadvantage of such approach was connected with the number of foreign keys and relations, but it was still a nice tradeoff.

Throughout this process, we made use of the AI tool called ChatGPT. We used it for identifying the errors in the SQL syntax as well as for the grammatical correctness of our report. We never used the output of the software without any modification. We made changes in SQL queries, made the sample records realistic, and ran every single SQL query in MySQL before submitting them.

We could have taken the project a step further with the help of some more time in hand. We would have made it more realistic with the help of some more functions like stored procedures, indexes, and triggers. Besides, we would have designed an interface that was linked to the database for managing appointments, prescriptions, and payments through a web application rather than the SQL queries alone. Overall, this project has made us learn more about the designing of databases, SQL execution and the importance of testing different components of a database.

GitHub repository link:

<https://github.com/abdulwahidAlQatari/Clinic-database>

Deliverables

1. **Project Report (PDF)** – System description, ER/EER diagram, relational schema, SQL code, screenshots, and reflection.
2. **SQL Script (.sql)** – Complete SQL implementation of the project.
3. **GitHub Repository** – Repository link with the complete project and commit history.
4. **Mid-Project Progress Report** – Summary of progress, challenges, decisions, and team contributions.
5. **Project Artifacts**
 - Live report document (with edit history): [link]
 - GitHub/GitLab repository (with commit history): [link]
 - Google Colab notebook (if applicable): [link]